

智能电商客服系统

——部署程序学习手册——

FastAPI + vLLM + RAG 的 LLM 应用全栈案例

写在前面

这份手册帮你读懂一个真实的 LLM 应用项目——一个基于大语言模型的电商客服系统。它不大, 代码量两千行左右,但它真实地展示了「一个 AI 系统在生产环境里需要解决哪些问题」。

读完这份手册,你应该能回答下面这些问题:

- ChatGPT 这类系统是怎么把模型变成「能用的产品」的?
- 为什么真实系统不只是「调一下大模型 API」就够了?
- 流式响应、向量检索、限流、鉴权,这些工程术语在代码里到底长什么样?
- 如果让我自己写一个类似的项目,我需要先想清楚什么?

怎么读这份手册

文档分四部分:

- **第一部分** 是程序分析,告诉你这个项目有什么、各部分负责什么、设计上的亮点,以及作者为了让你看清核心思想而做的简化。
- **第二部分** 是系统架构图,从「鸟瞰」的角度看整个系统由哪些组件构成。
- **第三部分** 是请求处理流程,顺着一次用户提问,把代码到底做了哪些事一步步拆开。
- **第四部分** 是学习路径,告诉你按什么顺序读源码、配套做哪些练习能真正掌握。

建议第一遍先快速浏览第一、二部分,建立整体印象;然后对照源码精读第三部分;最后跟着第四部分动手实践。

文档里的几种提示框

学习要点 出现这个标签的地方,是你应该重点记住的内容。

常见误区 这里讲的是初学者容易理解错的地方,看到要慢下来想一想。

思考一下 看到这个标签先别往下读,试着自己想想答案,再对照后文。

动手试试 这里有可以立即在代码上动手验证的小练习。

目录

第一部分 程序分析

- 1.1 这是个什么项目
- 1.2 目录里都有什么
- 1.3 系统怎么跑起来:start_all.sh
- 1.4 一次提问发生了什么
- 1.5 设计上的亮点
- 1.6 哪些地方做了简化
- 1.7 整体感受

第二部分 系统架构图

- 2.1 架构总览
- 2.2 四个层次分别在做什么
- 2.3 组件清单速查

第三部分 请求处理流程

- 3.1 流程图
- 3.2 关键节点详解

第四部分 学习路径

- 4.1 建议的阅读顺序
- 4.2 核心套路:LLM 应用都长这样



4.3 工程化:demo 和上线之间的距离

4.4 自己动手练习

4.5 一个反向追问的小技巧

附录 关键源文件速查表

第一部分 程序分析

1.1 这是个什么项目

这是一个基于大语言模型(LLM)的智能电商客服系统。把它放到现实场景里看,它要解决的是这样一类问题:

- 用户问「我昨天下的单啥时候到?」——客服系统得知道你的订单数据
- 用户问「七天无理由退货怎么操作?」——客服系统得知道平台的规则
- 用户接着问「那能改成顺丰吗?」——客服系统得记得前一句聊的是哪一单

这三个需求,对应三种不同的能力:查业务数据、查规则手册、记住上下文。光靠一个大模型是搞不定的——大模型不知道你的订单号,也不知道这家电商的最新政策。所以这个项目把三种东西拼起来:

- 一个微调过的大模型(负责理解问题、生成自然语言回答)
- 一套业务数据查询(订单、物流、退款、商品)
- 一个 PDF 知识库(产品手册、退换货政策这类文档)

学习要点 这种「LLM + 结构化数据 + 知识库」的组合,几乎是当前所有商用 LLM 产品的标准形态。理解了它,你就理解了 ChatGPT、Copilot、各种 AI 客服在企业里的实际部署方式。

用到的技术栈

类别	技术选型	在系统中的角色
推理引擎	vLLM	承载微调后的 Qwen1.5-1.8B-Chat 模型,提供

		OpenAI 兼容的流式接口
Web 框架	FastAPI	对外暴露 HTTP API,异步处理高并发请求
会话存储	Redis	存储多轮对话历史,同时作为限流计数器
向量数据库	ChromaDB	存储 PDF 知识库的向量表示,做相似度检索
嵌入模型	bge-m3	把文本转换成向量,供 ChromaDB 检索
前端	Gradio	提供对话界面和 PDF 上传管理
监控	Prometheus	采集 QPS、延迟分位数、错误率等指标

如果对其中某些名词还陌生,不用慌,后面会逐个讲清楚它们各自在系统里干什么活。

1.2 目录里都有什么

打开 deploy 目录,你会看到这样的文件结构:

```

deploy/
├─ config.py           # 全局配置,从环境变量读取
├─ gradio_app.py       # Gradio 前端,独立进程
├─ requirements.txt    # Python 依赖清单
├─ start_all.sh        # 一键启动脚本
├─ stop_all.sh         # 一键停止脚本
├─ 产品手册_电商平台.pdf # 示例知识库 PDF
└─ app/               # FastAPI 应用包
    ├─ main.py         # 入口、路由、请求编排
    ├─ auth.py         # API Key 鉴权
    ├─ ratelimit.py     # 滑动窗口限流
    ├─ session.py       # Redis 会话管理
    ├─ rag.py           # PDF 向量检索
    ├─ query_order.py   # 业务数据库查询
    └─ mock_db.py       # 模拟订单/物流/退款数据
  
```

注意「一个文件一个职责」这种组织方式。看文件名就知道它在做什么——auth 管鉴权、ratelimit 管限流、session 管会话。这是 Python 项目里最常见也最值得模仿的组织习惯。等你将来自己写项目时,也应该这样切分。

动手试试 在自己电脑上克隆下来,先 cat 一遍每个文件的开头几行注释,五分钟内就能对整个项目有个大致印象。不要急着读完,先建立地图。

1.3 系统怎么跑起来:start_all.sh

看完了项目目录,你可能会问:这些文件是怎么协同工作的?它们之间有启动顺序吗?

答案就在 `start_all.sh` 这个一键启动脚本里。整个系统由四个独立的进程组成,这个脚本按依赖关系把它们一个个拉起来,每一步都做幂等检查(已经在跑就跳过),并且每一步都等待健康检查通过再走下一步。读完这个脚本,你就理解了整个系统的「运行时拓扑」。

它做了什么

脚本主体是五个阶段,严格按依赖顺序自下而上:

阶段	做什么	端口
0. 准备	set -e 失败即停;cd 到脚本目录;创建 logs 目录	—
1. Redis	如果未安装则用 apt/dnf/yum 自动装;daemon 模式启动,限内存 512MB + LRU 淘汰	6379
2. vLLM	加载微调模型,首次加载约 2 分钟;开启前缀缓存加速;循环 30×5s 等 /health 就绪	8001

3. FastAPI	uvicorn 启动 2 个 worker;循环 10×2s 等 /health 就绪	8000
4. Gradio	起 gradio_app.py,等 3 秒认为就绪	7860

启动顺序的逻辑(关键)

这个顺序不是随便定的。每个服务的启动都依赖前面的服务已经就绪——倒过来启动,后起的服务找不到依赖,会一启动就崩。

```

Redis 必须先起      ← FastAPI 启动时要连 Redis 连接池
  ↓
vLLM 必须先起      ← FastAPI 健康检查会探 vLLM
  ↓
FastAPI 必须先起    ← Gradio 要调 FastAPI 的接口
  ↓
Gradio 最后起      ← 用户从这里进入
  
```

学习要点 「自下而上启动,自上而下停止」是分布式系统部署的通用规则——start_all.sh 启动顺序和 stop_all.sh 关闭顺序正好相反。

几个值得偷师的脚本工程习惯

习惯	这里怎么做的	为什么重要
幂等性	每一步都先检查服务是否已在跑,在跑就跳过	脚本可以反复执行,不会启动重复实例
健康等待	启动 vLLM 后用循环 ping /health,真就绪才走下一步	避免「进程在但还没响应」的假就绪状态
后台运行	nohup ... & 加日志重定向	脚本退出后服务继续跑,出问题去翻

		日志
可配置	<code>\${VLLM_PORT:-8001}</code> 这种带默认值的变量	用户可以用环境变量覆盖,不用改脚本
错误即停	<code>set -e + 每步 exit 1</code> 兜底	一处出错立即停止,避免连锁失败

vLLM 启动参数里的几个细节

第 2 阶段启动 vLLM 时传了几个参数值得专门讲一下,因为它们直接影响系统的性能和资源占用:

- `--gpu-memory-utilization 0.45` — 只用 45% 显存,给嵌入模型(bge-m3)和其他进程留空间
- `--max-num-seqs 64` — 同时最多处理 64 个请求,平衡吞吐和延迟
- `--max-model-len 2048` — 上下文长度上限,2K tokens 对客服场景够用
- `--enable-prefix-caching` — 开启前缀缓存。相同 prompt 前缀(比如固定的 system_prompt)的 KV cache 会被复用,大幅加速重复请求
- `--served-model-name customerservice` — 这就是为什么前面分析里说「模型对外名字叫 customerservice」

常见误区 看到 `workers=2` 别误以为是性能优化——实际上 uvicorn 两个 worker 各自加载一份 bge-m3 和 ChromaDB,内存翻倍。这正是 1.6 节会讲到的「有意简化」之一:为了部署简单牺牲了资源效率。

1.4 一次提问发生了什么

整个系统的核心入口是 `POST /v1/chat/stream`(定义在 `app/main.py` 第 89 行)。当用户发出一个

问题时,系统按下面这个顺序处理:

- **鉴权** — 从请求头读取 X-API-Key,与环境变量 `API_KEYS` 中配置的合法集合比对。校验失败立即返回 401,后续逻辑完全不执行。
- **限流** — 用 Redis ZSET 实现的滑动窗口算法。每个 API Key 每分钟最多 25 次请求,超过则返回 429。
- **拉取历史** — 用 `session_id` 从 Redis 读取最近 10 轮对话,作为模型的上下文。
- **检索编排**(这一步最值得注意)— 先调用 `query_order_context()` 识别用户是否在问订单、物流、退款、商品类问题;命中就走结构化的 `mock_db` 查询。如果业务库无命中,才退回到 `rag.search()` 查 PDF 知识库。
- **拼接 Prompt** — 把 `system_prompt`(系统人设)、检索结果(精确数据或知识片段)、历史对话、当前问题拼成一个完整的提示词,送给模型。
- **流式转发** — 用 `httpx` 以流式方式调用 `vLLM`,模型每生成一个 token 就通过 SSE 协议推送给客户端,客户端边收边显示。这就是 ChatGPT 那种「逐字出现」的效果。
- **异步落库** — 响应完全返回后,用 `asyncio.create_task` 在后台异步保存这一轮对话到 Redis。注意:用户感知到的延迟里不包括这部分写库时间。

学习要点 第 4 步「检索编排」的二级决策(业务库优先、RAG 兜底)是整个项目最值得记住的设计。它体现了一个原则:不要用大锤敲钉子——结构化数据该用 SQL 就用 SQL,RAG 只是兜底。

把上面这七步用流程图整体呈现一遍,你应该能看到主流程(左)和异步生成器(右)是怎么协作的:

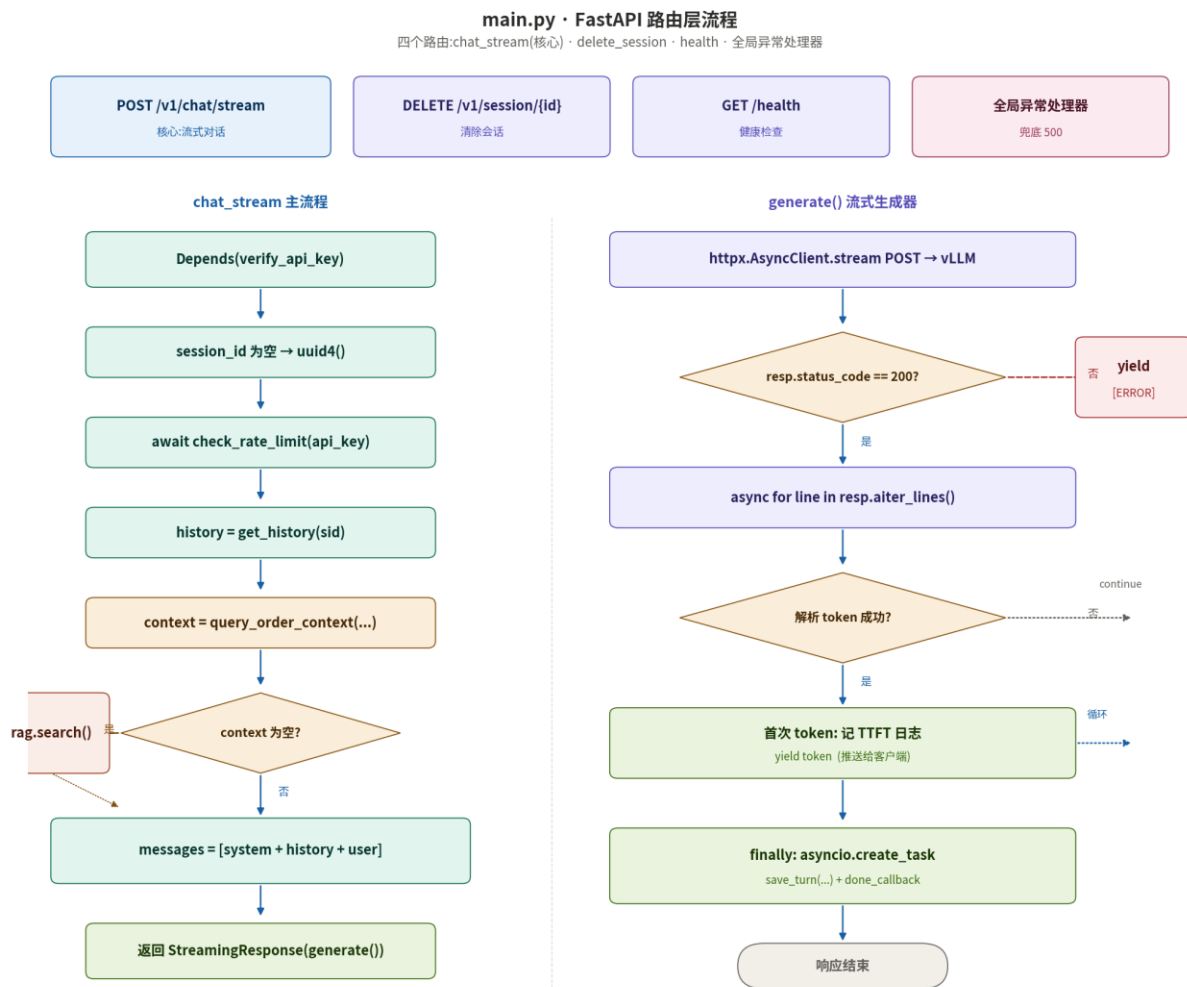


图 1-1 main.py · FastAPI 路由层流程

注意右半边的 generate() 是一个异步生成器——它不像普通函数一次性返回结果,而是边收 vLLM 的 token 边 yield 给客户端。这就是流式响应的底层机制。再注意右下角的「finally: asyncio.create_task」分支——它在主流程结束后异步把对话写回 Redis,带 done_callback 防止异常被吞。这是异步 Python 容易踩的一个坑,值得专门记一下。

1.5 设计上的亮点

query_order.py · 检索分流

这个文件做了一件看似简单但极其重要的事:用关键词集合 + 正则匹配,识别用户问的是「订单」「物流」「快递」还是「商品」,然后走对应的 mock_db 查询,最后把查到的结构化数据排版成「【系统检索结果】...」格式注入到提示词里。

为什么不能直接用 RAG?想象一下:用户问「订单 12345 到哪了」,如果走向量检索,系统可能在向量空间里匹配到「订单 12346」的物流记录——因为这俩订单号在向量空间里非常接近。这是灾难性的错误,因为客服系统绝对不能告诉用户错误的物流信息。

结构化查询和向量检索本质上是两种不同的工具:前者处理精确匹配,后者处理模糊匹配。它们必须分开,绝不能混用。

query_order.py 的内部流程是这样的:

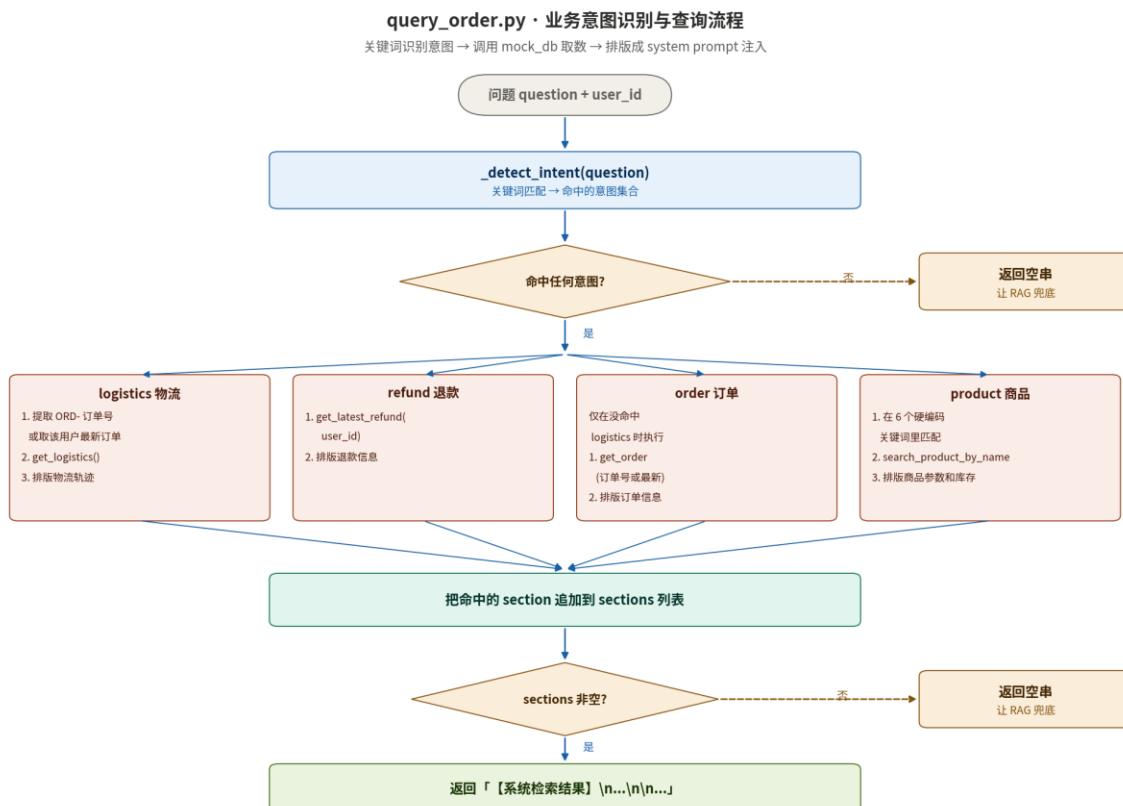


图 1-2 query_order.py · 业务意图识别与查询流程

注意这张图里四个意图卡片并排展示——同一个问题可能同时命中多个意图(比如「我的订单到哪了」既算「订单」也算「物流」)。所以这里用的是 set 而不是 if-else 链,命中的意图各自走自己的分支,最后把结果合并到 sections 列表里。这种「多意图并行处理」的写法比简单的 if-elif 链更接近真实业务的需求。

rag.py • 工程细节

RAG 模块里藏着几个值得学的工程习惯:

- 用「单例 + 双重检查锁」模式初始化嵌入模型和 ChromaDB 客户端,避免多并发请求时重复加载模型(嵌入模型加载很慢、占显存,绝对不能初始化两次)
- PDF 上传时做了路径白名单校验(`_is_safe_path` 函数),防止用户构造像 `../etc/passwd` 这类路径来读系统文件
- cosine distance 设了 0.6 的阈值,过滤掉相似度太低的「假命中」——宁可没结果,也不要错误结果

rag.py 有两条独立的流程——入库(左)和检索(右),共享同一个嵌入模型单例和 Chroma 客户端。

看下面这张并排图就能一眼分清这两条路径:

rag.py · PDF 入库与向量检索流程

两个对外功能: ingest_pdf 入库、search 检索; 共享一个嵌入模型单例和 Chroma 客户端

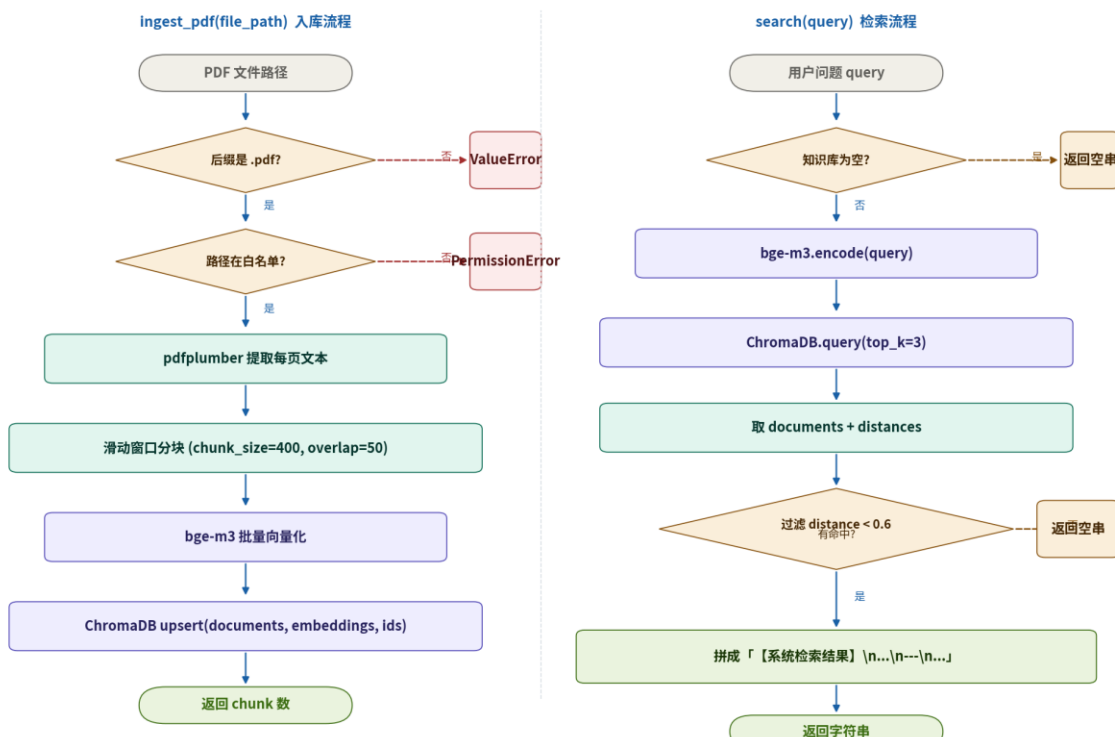


图 1-3 rag.py · PDF 入库与向量检索流程

两条流程都做了「失败早退」处理——入库时扩展名或路径不对就抛错,检索时知识库为空或没命中阈值就返回空串。返回空串这个设计很关键:它不是错误,而是让上层主流程知道「这里没结果,该用别的兜底了」。这套「失败安静返回、让上层决定」的模式在工程上非常实用。

session.py · 初始化锁

Redis 连接池的创建用 `asyncio.Lock` 包裹,加双重检查模式,保证在多 worker、多并发场景下不会创建出多个连接池。这是一个看起来不起眼但非常关键的细节——很多线上事故就是因为「每次请求都建一个新连接」导致的连接泄漏。

ratelimit.py · 降级策略

当 Redis 不可用时,这个模块选择放行请求而非拒绝。这是一个明确的「可用性优先」决策——宁

可让攻击者短时间内绕过限流,也不能让所有正常用户都用不了服务。

思考一下 上面这个决策合理吗?如果你是这家公司的工程师,你会选放行还是拒绝?在继续往下读之前,先自己想想理由。

config.py • 生产保护

当环境变量 `PRODUCTION=true` 时,系统强制要求必须配置 `API_KEYS`,否则启动失败。这种「生产环境检查」机制避免了「忘了配置就上线」这种低级事故。

1.6 哪些地方做了简化

没有完美的代码。但更重要的是,这个项目作为一个教学和演示用的样板,在很多地方做了「有意的简化」。

这一点你需要从一开始就理解清楚:下面这份清单不是在说「这代码写得不好」,而是在说「为了让你看清楚我们真正想突出的东西,这些次要的细节被故意省略了」。如果把每一条都补全——把 `mock_db` 换成真实数据库、把意图识别换成 `NER` 模型、把鉴权换成完整的 `OAuth2`、把所有边缘情况都处理掉——代码量会膨胀到一万行以上,你就再也看不清「LLM 应用的核心套路」长什么样了。

换句话说:这些简化是教学的成本,也是教学的礼物。它们让你能在两千行代码里读懂整个系统;同时它们也提醒你——真实生产中你需要把哪些地方补回去。

下面这张表列出了主要的简化点。第一列是简化的类别,第二列是具体做了什么妥协,第三列告诉你这件事如果做完整需要补什么——这一列是给你将来做自己项目时的参考。

类别	当前的简化做法	生产环境应该怎么补
----	---------	-----------

安全:CORS	CORS_ORIGINS 默认允许任何源(*),仅 warn 提示	配置具体的前端域名白名单
安全:路径校验	_is_safe_path 把所有含 /tmp/ 的路径放行,简化了校验逻辑	用更严格的路径前缀检查,只允许指定的上传目录
限流降级	Redis 异常时完全放行(可用性优先),让 demo 不会因为 Redis 故障就跑不起来	改为「拒绝」并配上熔断器、本地兜底计数
限流原子性	先 zcard 判断、再 zadd 记录两步操作,代码更直观	用 Lua 脚本或 pipeline 保证原子性
进程边界	Gradio 进程直接 import rag 模块,部署更简单(一个 docker 起两个端口)	Gradio 通过 HTTP 调 API,模型只在 FastAPI 进程里加载一次
业务数据	mock_db 是内存字典,无需配数据库就能跑起来	替换成 PostgreSQL/MySQL + SQLAlchemy ORM + 连接池
意图识别	商品关键词写死成 6 个中文词,逻辑清晰可见	改用 NER 模型或让 LLM 做 Function Calling
错误处理	用 order['order_id'] 直接访问字段,代码更短	全部改成 .get() 并处理缺失字段
健康检查	用字符串 replace 拼接 vLLM 健康检查路径	用正经的 URL 解析库,配置可配的 health 路径
历史写入	save_turn 三步操作不是原子的,代码更易读	用 Redis 事务或 Lua 脚本保证原子写入

学习要点 记住:一个好的教学项目不是「最完整」的项目,而是「最能让你看清核心思想」的项目。当你看到这些简化时,不要批评作者,而要问自己——「如果是我写,我能在保持简洁的同时把哪一条补好?」这才是有用的学习视角。

常见误区 看到「不足清单」就觉得这套代码不能用,这是初学者的典型反应。事实上,在 demo 和教学场景里,这些简化是正确的;到了生产场景,这些地方就需要补全——但补全的前提是你已经理解了原本的设计意图。先理解,再扩展。

1.7 整体感受

综合来看,这是一个结构清晰、工程化程度较高的 LLM 应用样板。它的几个突出优点:

- 分层得当——鉴权、限流、检索、会话、推理各自一个文件,职责单一
- 关注流式响应的细节,包括首 token 延迟(TTFT)日志和后台任务异常回调
- 知道在 RAG 之外做业务数据查询,这是大多数 demo 都没意识到的关键设计
- 有健康检查、Prometheus 指标接入、统一异常处理等可观测性要素

它的定位是「演示 + 中小规模部署」,不是「直接上生产」。mock_db 是内存字典而非真实数据库,意图识别靠关键词,商品搜索是硬编码列表。这些都在源码注释里坦诚标注「真实生产中替换为 SQL/ORM 即可」,作者很清楚自己的边界。

结合 1.6 节再强调一遍:这种「不求全」的设计反而是这个项目作为学习材料的最大优点。如果作者一开始就把 mock_db 写成完整的 PostgreSQL 接入、把意图识别换成 BERT 模型、把鉴权改成 OAuth2 全套流程,你打开代码看到的会是一堆框架代码,反而看不到「LLM 应用到底新在哪里」。把次要的复杂性砍掉,核心思想才能浮出水面——这是好的教学材料的共性。

如果将来你要把它推向生产,优先级最高的几件事是:CORS 收紧、Redis 降级改成「拒绝」而非「放行」、Gradio 通过 HTTP 调 API 而非直接 import、把 mock_db 换成真实数据源、增加请求日志脱敏。

第二部分 系统架构图

2.1 架构总览

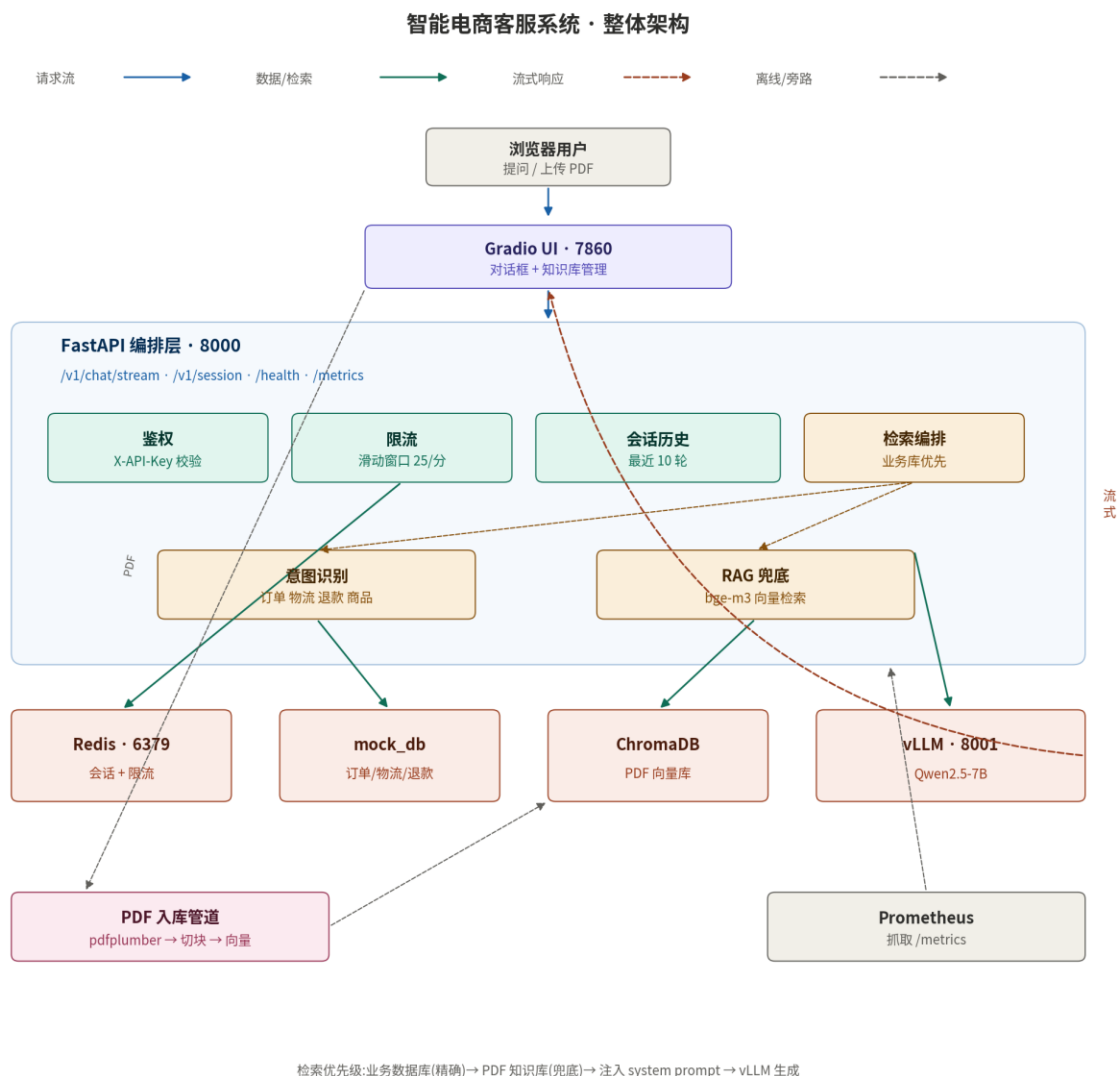


图 2-1 智能电商客服系统整体架构

2.2 四个层次分别在做什么

整个系统从上到下可以分为四层。理解这四层的职责划分,是理解整个系统的钥匙。

第一层 • 展示层(Gradio)

Gradio 运行在 7860 端口,提供两个核心功能:一是对话框,用户输入问题、看到流式回复;二是知识库管理,允许上传 PDF 并入库。

Gradio 进程是独立的,它通过 HTTP 调用 FastAPI(目前的实现里部分功能通过直接 `import` 调用,这是前面提到的一个改进点)。这种解耦让前端可以随时换成微信、网页、App 等任何形态——只要协议不变。

第二层 • 编排层(FastAPI)

这是系统的「大脑」,运行在 8000 端口。它本身不存数据、不做推理,但它决定数据怎么流、什么时候调用谁。具体它负责四件事:

- 守门——鉴权和限流是请求进入系统的两道闸门
- 记忆——从 Redis 拉取会话历史,作为上下文
- 决策——意图识别让请求走 `mock_db` 还是 RAG
- 传话——把整理好的提示词转发给 vLLM,把 vLLM 的流式输出回传给客户端

学习要点 编排层的核心价值在于「决策」而非「计算」。所有耗资源的计算(向量相似度、模型推理)都委托给了别的服务,FastAPI 自己只做轻量的 I/O 操作。这就是它能高并发处理请求的根本原因。

第三层 • 数据层(Redis + mock_db + ChromaDB)

三种数据,三种存储,对应三种不同的访问模式:

数据类型	存储	访问模式	为什么用这个
------	----	------	--------

会话历史	Redis	按 session_id 键查找,TTL 自动过期	需要跨进程共享 + 自动清理
业务数据	mock_db	按订单号、用户 ID 等键查找	结构化精确查询(真实场景应换数据库)
知识库	ChromaDB	按向量相似度搜索	处理「语义相近」的模糊匹配

第四层 • 推理层(vLLM)

vLLM 是一个专门为 LLM 推理优化的引擎,运行在 8001 端口。它实现了 OpenAI 兼容的 API,所以 FastAPI 调用它的方式和调用 ChatGPT 几乎一样。

vLLM 加载了微调过的 Qwen1.5-1.8B-Chat 模型,模型名叫 customerservice。微调让模型在客服场景下表现得比通用模型更好——更礼貌、更专业、更愿意承认「我不知道」。1.8B 参数规模在小显存机器上也跑得动,适合作为演示和学习用的轻量模型。

旁路 • PDF 入库管道与监控

PDF 入库是离线的:用户上传 PDF 后,系统用 pdfplumber 提取文字,切成 chunks,用 bge-m3 编码成向量,存入 ChromaDB。这个过程不需要 LLM 参与,完全独立。

Prometheus 在后台定期抓取 FastAPI 的 /metrics 端点,采集 QPS、延迟分位数、错误率等指标。一句话:在线服务必须有监控,这是底线。

2.3 组件清单速查

层级	组件	端口	核心职责
展示	Gradio UI	7860	对话界面 + 知识库管理

编排	FastAPI	8000	鉴权、限流、检索编排、流式转发
编排	鉴权模块	—	X-API-Key Header 校验
编排	限流模块	—	Redis 滑动窗口,25 次/分钟
编排	意图识别	—	关键词匹配订单/物流/退款/商品
编排	RAG 检索	—	bge-m3 向量化 + ChromaDB 相似度搜索
数据	Redis	6379	会话历史 + 限流计数 + TTL
数据	mock_db	—	内存字典模拟业务库
数据	ChromaDB	—	PDF 知识库向量存储
推理	vLLM	8001	Qwen1.5-1.8B-Chat 流式生成,OpenAI 兼容协议
监控	Prometheus	—	抓取 /metrics 端点

看完了「系统由什么组成」,接下来要看的是「一个请求来了系统在做什么」——这是下一部分的主题。架构图是静态视角,流程图是动态视角,两者结合才能完整理解一个系统。下一部分会以 POST /v1/chat/stream 接口为例,把一个完整的请求处理过程拆解到每个步骤。

第三部分 请求处理流程

3.1 流程图

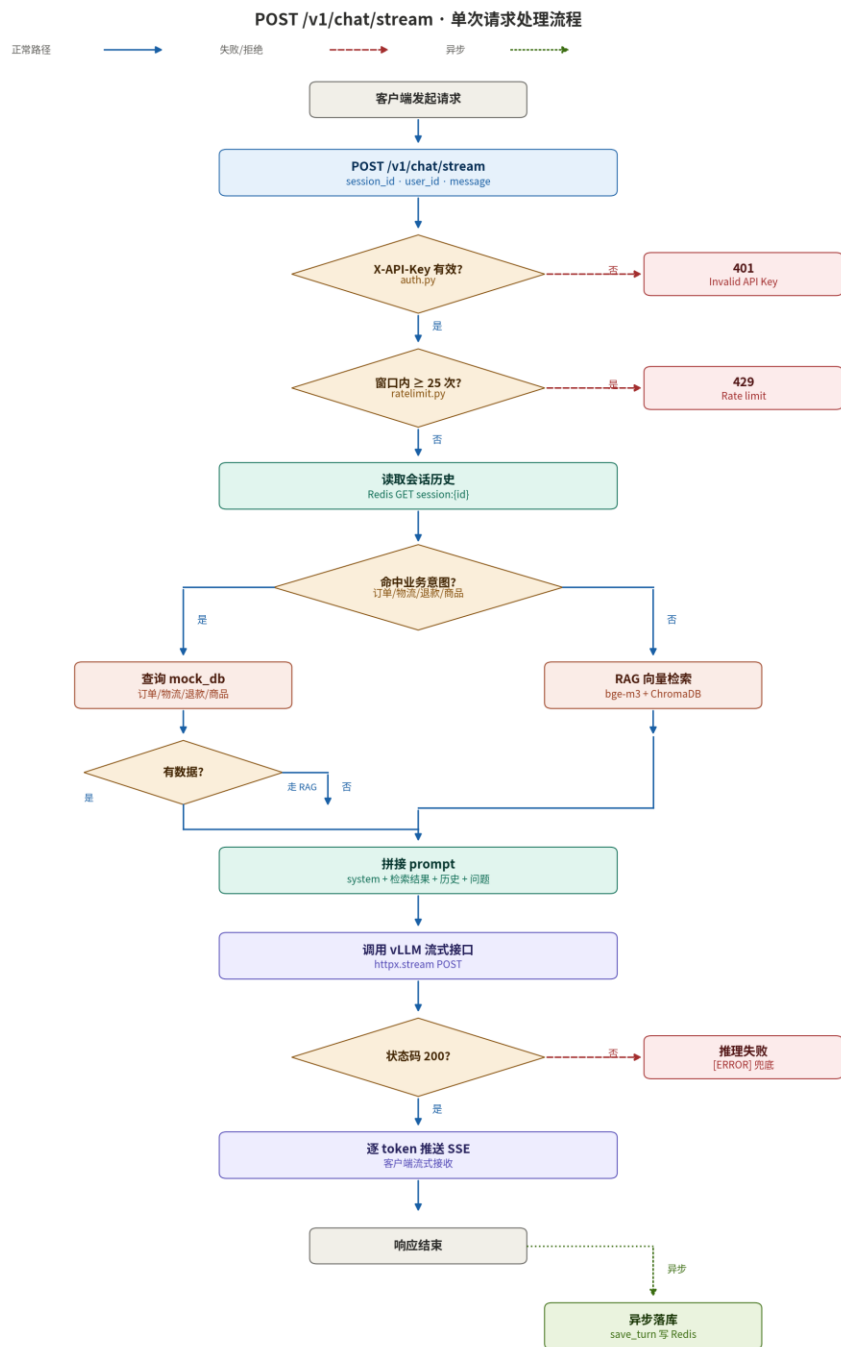


图 3-1 POST /v1/chat/stream 请求处理流程

3.2 关键节点详解

两道闸门:鉴权 + 限流

请求一进来,先过两道闸门。这两步必须串行、必须前置——任一失败立即短路返回,后续所有逻辑(包括最贵的 LLM 调用)都不执行。

鉴权用的是最简单的 **API Key** 方案,适合服务端到服务端的调用。如果是直接面向终端用户,可能还需要 OAuth2/JWT 等更复杂的方案。

限流的算法值得专门看一下。Redis ZSET 滑动窗口的精妙之处在于:用时间戳作为 score,过期的记录可以一条命令删除,无需后台清理任务。这是个能让你眼前一亮的小技巧——以后做自己的项目也能用得上。

常见误区 Redis 不可用时限流模块会放行而非拒绝。这是「可用性优先」的选择,但攻击者只要打挂 Redis 就能绕过限流。这种权衡在工业界很常见——选择没有对错,但你要理解每种选择的代价。

检索的二选一:业务库 vs RAG

这是整个流程最有设计感的一步。意图识别命中关键词就走 mock_db 做精确查询,失败或没命中就走 RAG 做模糊检索。两条路最终汇到同一个提示词拼装节点。

举几个具体例子帮你理解:

- 用户问「订单 12345 啥时候到」→ 命中「订单」关键词 → 查 mock_db → 注入精确的订单状态

- 用户问「七天无理由怎么操作」→ 没命中任何业务关键词 → RAG 检索 PDF → 注入相关政策片段
- 用户问「订单 99999 怎么办」→ 命中「订单」→ 查 mock_db 但没数据 → 退回 RAG → 用知识库兜底

流式回路:为什么 ChatGPT 让你感觉很快

从「调用 vLLM」到「逐 token 推送」这一段,看上去是两个步骤,实际上是一个嵌套的异步循环。

FastAPI 通过 `httpx.stream` 打开一个长连接,vLLM 每生成一个 token 就推一行 SSE 数据,FastAPI 立刻 `yield` 给客户端,客户端立刻渲染到屏幕。

总延迟的体感差异是数量级的:一个 500 字的回答如果等全部生成完(大约 5 秒)再返回,用户面对 5 秒空屏;如果流式返回,200 毫秒用户就看到第一个字,边读边生成。这就是 ChatGPT 给人的「快」的来源——其实总时间没变,但用户体感完全不同。

异步落库:用户感知不到的尾巴

响应结束后,会话历史不是同步保存的,而是通过 `asyncio.create_task` 异步执行的。这意味着客户端在收到最后一个 token 之后就可以断开连接,Redis 写入在后台慢慢做。

这个细节体现了对延迟敏感度的理解:能放后台的工作就不要阻塞响应。虽然单次 Redis 写入只要几毫秒,但累加起来对 P99 延迟有可观的改善。

学习要点 源码里 `asyncio.create_task` 之后还绑定了 `done_callback`,这是因为 `asyncio` 的 Task 如果没人引用,异常会被悄悄吞掉。绑定回调可以在异常时打日志,这是 `async Python` 的一个著名坑点,你以后写异步代码也要小心。

第四部分 学习路径

前三部分已经把这个系统讲清楚了。这一部分是给你的一份「怎么学」的建议:按什么顺序读代码、重点关注哪些套路、做哪些练习能真正掌握。

4.1 建议的阅读顺序

不要打开 main.py 就一行一行读到底——那样你会很快迷失在细节里。下面是一个被验证过有效的阅读顺序:

步骤	文件	重点关注什么
1	README + start_all.sh	整个项目怎么跑起来、依赖哪些服务(配合本手册 1.3 节)
2	config.py	有哪些可配置项,看到的环境变量大致说明系统在意什么
3	本手册第二、三部分	对照架构图和流程图,在脑子里建立索引
4	main.py	看路由定义和 chat_stream 主流程,先看骨架不看细节
5	auth.py、ratelimit.py	理解请求是怎么被守门的
6	session.py	理解状态怎么跨请求保持
7	query_order.py + mock_db.py	理解业务数据怎么被引入提示词
8	rag.py	理解 RAG 的完整流程:切块 → 向量化 → 检索
9	gradio_app.py	看前端怎么调后端

10

回头精读 main.py

这时你已经知道每个 import 的来龙去脉了

动手试试 找一个完整的下午(三到四小时),按上面的顺序边读边在纸上画依赖关系。读完以后试着关掉源码,自己复述出整个流程——这比读十遍但不复述更有效。

4.2 核心套路:LLM 应用都长这样

读完这个项目你应该带走两个最重要的认知。

套路一:LLM 应用的本质是「提示词拼装」

看完 main.py 里拼提示词的那几行,你应该意识到一件事:LLM 应用工程师 90% 的工作就是决定往提示词里放什么。

这套系统的答案是:

```
system_prompt(人设和规则)
+ 检索结果(业务数据 or 知识库片段)
+ 历史对话(最近 N 轮)
+ 当前用户问题
```

这是个非常通用的模板,几乎所有 RAG 应用都长这样。掌握了这个模板,你就掌握了构建 LLM 应用的基本骨架。以后你写任何类似的项目,都可以从这个模板出发。

套路二:检索的二选一——精确数据 vs 模糊知识

这是这套程序最值得记住的设计决策。看下面这张对比表:

维度	业务数据(订单/物流)	知识库(PDF)
数据形态	结构化、键值精确	非结构化、语义匹配

检索方式	SQL / 字典查询	向量相似度
不能错的程度	完全不能错	模糊也能用
用关键词识别意图	适合	不适合

学习要点 RAG 不是万能锤子。问「订单 12345 状态」如果走向量检索可能找错单子,这是灾难。工业界的常见做法是:先用关键词识别意图、走结构化查询,失败再用 RAG 兜底。这个分层思路应该刻进你的设计直觉。

套路三:流式响应

用户感觉「快」不一定是真的快——首字节时间(TTFT)远比总时间重要。这个项目里整套 async/await + SSE 的代码看上去复杂,本质上就是为了让用户尽早看到第一个字。

你以后做任何用户可见的 LLM 应用,默认就应该上流式——不流式的 LLM 产品在体感上和能流式的差距是数量级的。

4.3 工程化:demo 和上线之间的距离

「能跑」和「能上线」是两回事。这一节列的几个点是 demo 和生产之间最重要的那段距离,值得你单独想清楚。

鉴权 + 限流——任何对外服务的标配

X-API-Key 是最简单的鉴权方案。对比 OAuth2/JWT 的复杂度,你能感受到「简单优先」的价值——能用 50 行解决的事,不要写 500 行。

限流的算法看似简单,但背后的取舍值得思考:用滑动窗口比固定窗口公平,用 Redis 比内存计数更适合多机部署。

思考一下 Redis 挂了,限流该放行还是拒绝?这是可用性 vs 安全性的经典权衡,没有标准答案。试着自己列出每种选择的利弊。

会话状态——为什么不能存在内存里

想想看:如果用 Python 字典存会话,会有什么问题?

- 重启服务,所有用户的对话历史全丢
- 多进程或多机部署时,不同实例之间不共享
- 没有自动过期机制,内存会一直涨

由此引出 Redis 的三个核心价值:**持久化、跨进程共享存储、TTL 过期**。看 session.py 时重点关注它是怎么用这三个特性的。

异步编程——为什么整套代码都是 async/await

LLM 应用是 **I/O 密集型**——大部分时间在等 vLLM 返回。如果用同步代码,一个请求把整个进程占满,QPS 上不去。

一个具体例子:main.py 里响应结束后用 `asyncio.create_task` 异步保存历史。用户拿到回答就立刻断开连接,写库在后台慢慢做——总延迟里不算这部分时间。这种细节才是工程师水平的体现。

监控——服务上线后你怎么知道它活着

Prometheus 的 /metrics 端点暴露:QPS、P50/P95/P99 延迟、错误率、token 生成速度。这些指标在出事的时候是你唯一的「眼睛」——没监控的服务等于裸奔。

4.4 自己动手练习

看懂了不等于学会了。下面这些练习按难度从低到高排列,选你能力范围内的去做。完成最难的两

个,你基本可以说自己掌握了这套程序。

难度	练习	目标
★	修改 system_prompt,把客服人设从「小智」改成别的形象,对比对话风格的变化	体会提示词的威力
★★	上传一份新 PDF,问知识库相关问题,观察 RAG 检索返回的片段	理解向量检索的实际效果和阈值的作用
★★	给 mock_db 加一条新订单,从前端问出来	走通业务数据流程
★★★	加一个新的业务意图,比如「查会员等级」,从意图识别到 mock_db 到提示词全链路自己实现一遍	理解模块协作
★★★★	把限流从「每分钟 25 次」改成「按 token 数限流」,体会更细粒度的资源控制	深入限流算法
★★★★★	把 mock_db 换成真实 SQLite,接上 SQLAlchemy ORM,加上连接池	把 demo 推向生产

动手试试 做练习的时候,先不要看任何参考实现。卡住了就去翻已有代码找相似的写法,模仿它。先做出能跑的版本,再看怎么改进。这比抄一份漂亮代码学到的多得多。

4.5 一个反向追问的小技巧

读完每个模块,问自己一句:如果没有它会怎么样?

这个反向追问能帮你从「记住」升级到「理解」:

- 没有鉴权 → 谁都能白嫖你的 LLM,一夜账单破产

- 没有限流 → 一个脚本能把账单刷爆
- 没有 Redis → 多进程部署上下文全乱
- 没有 RAG → 客服一本正经胡说八道
- 没有意图识别 → 订单查询变成玄学
- 没有流式 → 用户等到怀疑人生
- 没有监控 → 出了事故只能用户来告诉你

当你以后做自己的项目时,问问自己「我有没有这些东西」——少哪个就该补哪个。这是把「知识」变成「判断力」的关键。

附录 关键源文件速查表

读源码时随时翻一翻这张表,能帮你快速定位每个文件的作用:

文件	行数(约)	作用
app/main.py	270	FastAPI 入口,定义所有路由,组装请求处理流程
app/auth.py	20	API Key 鉴权,最简单的 Header 验证
app/ratelimit.py	70	Redis ZSET 滑动窗口限流
app/session.py	120	Redis 会话管理,异步连接池
app/rag.py	200	PDF 入库 + 向量检索,bge-m3 + ChromaDB
app/query_order.py	240	业务意图识别 + mock_db 查询
app/mock_db.py	180	内存字典模拟订单/物流/退款/商品数据
config.py	180	全局配置,从环境变量读取所有参数
gradio_app.py	280	Gradio 前端,对话框 + PDF 管理
start_all.sh	—	一键启动 Redis → vLLM → FastAPI → Gradio,详见 1.3 节
stop_all.sh	—	一键停止所有服务,关闭顺序与启动相反

— 学习手册结束 —

祝你学有所得